Homework 2

NERS/ENGR 570 Fall 2026

Due October 4, 2026 by 11:58 pm
(Target Intermediate Deadline: Complete Exercises 1 and 2 by 9/30)

Introduction

One of the main motifs of scientific computing is sparse linear algebra. Sparse linear systems commonly arise from the discretization of partial differential equations. Because of their sparsity, it is often best to choose a way to store the data so that only the non-zero values are stored in memory. Over the years, several sparse matrix storage formats have been devised. Many scientific software libraries with capabilities of solving sparse linear systems, make use of these formats. In this homework you will implement several sparse matrix storage formats, along with a sparse matrix-vector multiplication kernel for each format. The learning objectives of this assignment are the following:

1. (Knowledge/Skill) reinforce and apply the concepts of index mapping in the context of linear algebra
2. (Knowledge/Skill) be able to work with several sparse matrix storage formats
3. (Skill) Continue our practice with high-level programming languages.
4. (Skill) Analyze the experimental performance of matrix data structures

This assignment will also help to form the foundation of future assignments and concepts related to testing and software design.

Matrix Market

For real world examples of matrices arising in engineering we will be working with data from the National Institute of Standards and Technology (NIST) Matrix Market. (https://math.nist.gov/MatrixMarket/)
The specific matrices that we will use as sample data in each of the exercises are:

- NNC666 https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/nucl/nnc666.html
- S1RMQ4M1 https://math.nist.gov/MatrixMarket/data/misc/cylshell/s1rmq4m1.html
- IMPOL E https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/chemimp/impcol_e.html
- FS 183 1 https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/facsimile/fs_183_1.html
- LNS 3937 https://math.nist.gov/MatrixMarket/data/Harwell-Boeing/lns/lns_3937.html

Note that Matrix Market also provides source code in C and Fortran for reading the matrix formats. You are welcome (perhaps encouraged) to use this code to load the matrix into memory in your programs. You should then take that data and convert it into the one of the required sparse matrix formats for the SpMV kernels you will implement.

In this assignment you may work in your preferred language of C, C++, or Fortran. You are not permitted to use any other libraries or code in your implementation.

Program Requirements

Write a program such that it can process the following command line arguments, and perform a fixed number of matrix-vector multiplication using a given matrix file, vector file, and matrix format.

Usage: `SpMV.exe <spfmt> <nmults> <mmfile> <vecfilein> <vecfileout>`

`<spfmt>` The sparse matrix format keyword. One of:

- DEN – dense matrix (see exercise 1)
- COO – coordinate format (see exercise 2)
- CSR – compressed sparse row (see exercise 3)
- ELL – ELLPACK format (see exercise 4)
- JDS – jagged diagonal storage (see exercise 5)

`<nmults>` The number of matrix-vector multiplications to perform (for timing purposes)

`<mmfile>` Name of the matrix market file to read in the matrix from

`<vecfilein>` File name for the input vector (we will supply our own input to the autograder)

`<vecfileout>` File name to output the result of the matrix vector multiplication (this is what we will be comparing in the autograder).

Mathematically, we may write

$$A^n x = b$$

where `<mmfile>` is the data for A, `<vecfilein>` is the data for x, and the result, b is written to `<vecfileout>`. The number of matrix vector multiplications, `<nmults>` corresponds to n.

Perform all calculations in double precision.

Exercise 1 (10 pts) – Dense Matrices

To motivate the use of sparse matrix formats, we ask that you first provide a simple matvec implementation using fully dense matrices. In this exercise allocate A as a two-dimensional matrix, and initialize all the values to 0. Then, load the coefficients from the file into this array to perform the matvec.

Exercise 2 (15 pts) – COO Format

The COOrdinate format (COO) was covered in class. For each non-zero value, the COO format stores the row index, column index, and value in three arrays: `rowIdx(:)`, `colIdx(:)`, and `value(:)`, as illustrated in the figure below.

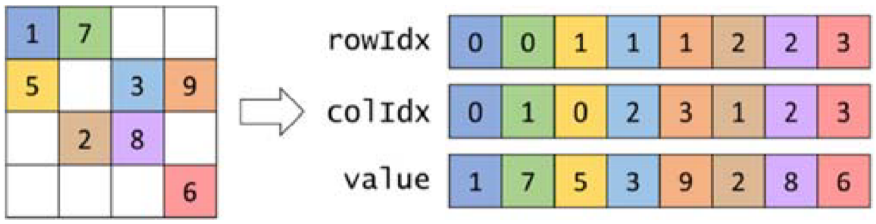


Figure 1: Example of the COO format

Exercise 3 (15 pts) – CSR Format

The compressed sparse row format (CSR) is similar to COO, except instead of storing the row index of each non-zero element, these non-zero values are grouped by row. For example, we store the nonzero elements of row 0 (1 and 7) first, followed by the non-zero elements of row 1 (5, 3, and 9), followed by the nonzero elements of row 2 (2 and 8), and finally the nonzero elements of row 3 (6). The `rowPtrs` array stores an index into the `colIdx` array where the first non-zero element in that row is found. This is known as indirect indexing, where one index (e.g., a loop counter) is used to retrieve a different index from some other data structure (e.g., the `rowPtrs` array in this context). For a matrix with M rows, the `rowPtrs` array stores $M+1$ values so that the number of elements in row `i` may be determined with `rowPtrs[i+1] - rowPtrs[i]`.

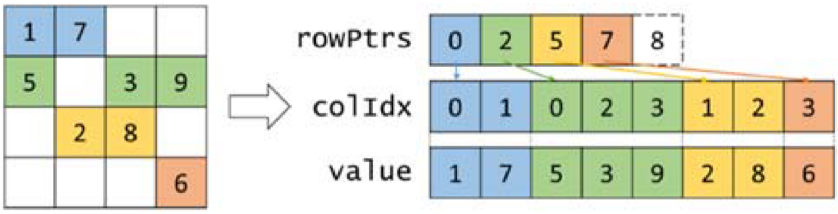


Figure 2: Example of the CSR format

Exercise 4 (15 pts) - ELL Format

In some cases, performance can be improved by applying data padding and transposition on the sparse matrix data. These ideas were used in the ELL storage format, whose name came from the sparse matrix package in ELLPACK, a package for solving elliptic boundary value problems. A simple way to understand the ELL format is to start with the CSR format, as is illustrated in the figure below. This matrix format is represented using a `colIdx` and `value` array.

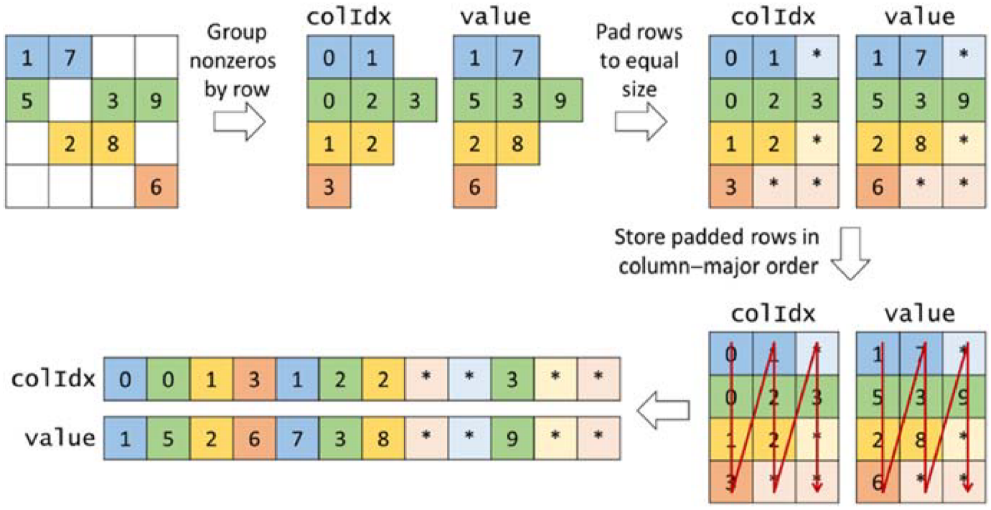


Figure 3: Example of the ELL format

Exercise 5 (20 pts) - JDS Format

Another format which can improve performance, but without the need to perform any padding, is the JDS format. The idea is to sort the rows according to their length, say, from the longest to the shortest. Since the sorted matrix looks largely like a triangular matrix, the format is often referred to as the jagged diagonal storage (JDS) format.
As we sort the rows, we typically maintain an additional `row` array that preserves the index of the original row. Whenever we exchange two rows in the sorting process, we also exchange the corresponding elements of the row array. Thus we can always keep track the original position of all rows. After the rows have been sorted, the non-zeros in the `value` array and their corresponding column indices in the `colIdx` array are stored in column-major order. A `iterPtr` array is added to track the beginning of the nonzero elements for each iteration. This storage format has the most complex index mapping of the formats presented in this assignment.

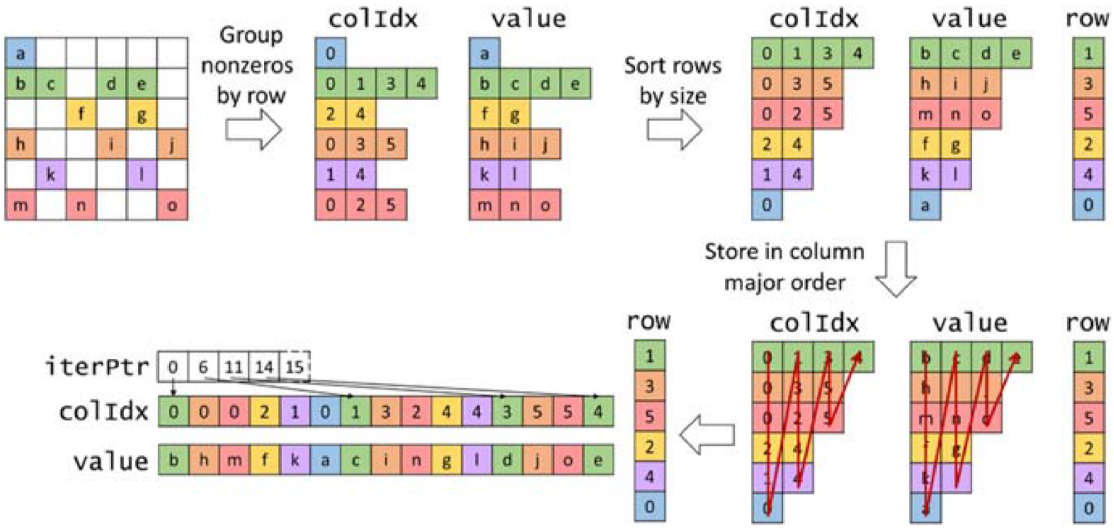


Figure 4: Example of the JDS format

Exercise 6 (25 pts) - Questions and Reflection

1. For the S1RMQ4M1 matrix, which format do you think will be most performant? Explain why you think this in at least a few sentences. Was your prediction correct? If not, in a few sentences explain what detail in your original thinking may have been absent or misunderstood.
2. For the FS 183 1 matrix, which format do you think will be most performant? Explain why you think this in at least a few sentences. Was your prediction correct? If not, in a few sentences explain what detail in your original thinking may have been absent or misunderstood.
3. For each of the sample matrices used, do you think there exists another format that might be more performant than the ones implemented? Explain why in a few sentences, identifying the characteristics of the format that would make it more performant.
4. Describe any function interfaces you wrote in your code (e.g. the structure) and why you chose those interfaces. For example,

```
INTEGER FUNCTION process_command_line_args(commandLine) RESULT(ierr)
    CHARACTER(LEN=*) :: commmandLine
    INTEGER :: ierr
END FUNCTION
```

I used the following routine to process the command line arguments and return an error code. I did so because this let me separately implement and test the command line processor from the functional code. This is a good idea because they are two inherently different functional capabilities of the program with non-overlapping requirements.

5. For each of the sample matrices and storage formats, calculate the amount of memory in KiB, MiB, or GiB for fully dense storage, and for sparse storage, and put them in a table. Show the equations you used to calculate the memory usage.

6. Using your program, generate a table of run times for each matrix and storage format. The time reported should correspond to the average time of the SpMV operation only (not any I/O). Use an `nmults` value that is sufficiently large (at least at least a few hundred) to ensure the SpMV timing is accurate.

Extra Credit 1 (5 pts) - TACO

Implement a CSR SpMV kernel using the Tensor Algebra Compiler, TACO (http://tensor-compiler.org/). Include the timing results of the TACO CSR for each matrix in your table for Exercise 6. Was the TACO CSR more performant than your other implementations? In a few sentences, explain why it was or was not.

Submission

Upload your source code and a `Makefile` in a tarball to build and test your executable within the Autograder.

The following command will be used to extract and compile your code:

```
$ tar -xzvf hw2.tgz && make
```

Your code will be graded using commands such as:

```
$ ./SpMV.exe DEN 1 fs_183_1.mtx fs_183_1.vecin fs_183_1.student
$ python3 compare_vecs.py fs_183_1.student fs_183_1.vecout
```

Additionally, upload to Canvas a single PDF containing:

- Written answers and tables for the questions corresponding to Exercise 6.
- An appendix of your source code.

The Autograder will supply a reference vector to your code to multiply with a matrix. The solution computed by your code will be compared with a reference. If the ∞-norm of the difference in the reference solution and your solution deviate by more than 1e-4, it will not be counted as correct. Vector inputs, outputs, and comparison script to test the correctness of your implementation may be found at https://github.com/bkochuna/ners570-hw2.git